



AUTONOMOUS SOC PLATFORM

Aegis Security

Autonomous Incident Investigation & Response at Machine Speed

An enterprise-grade Security Operations Center platform powered by a multi-turn ReAct reasoning agent. Detects, investigates, contains, and verifies cybersecurity threats end-to-end — without human intervention — in under 3.2 seconds.

 github.com/Sunil56224972/Agies-Security

▶ Watch Demo Video (MP4)

INFERENCE ENGINE

Groq LPU ·
LLaMA 3.3
70B

DATABASE

Neon
Serverless
PostgreSQL

STANDARDS

MITRE
ATT&CK ·
NIST SP 800-
61

LICENSE

MIT · Open
Source

The Autonomous SOC Problem

Why alert fatigue demands a self-directing agent — not another SOAR playbook.

🕒 Problem Statement

Modern enterprises face **alert fatigue** from high-velocity NIDS sensors (Snort, Suricata) generating hundreds of events per hour. Human analysts must manually correlate alerts with CMDB records, cross-reference CVE databases, parse endpoint logs, and enforce firewall containment — a process that takes **20–45 minutes per incident**. During this window, adversaries move laterally, deploy ransomware, and exfiltrate data.

👤 Target Users

Tier-1 / Tier-2 SOC Analysts
overwhelmed by false positives who need intelligent triage automation to focus exclusively on confirmed threats.

Incident Response Teams requiring fast, auditable containment with full MITRE ATT&CK traceability.

Enterprise Security Operations
seeking sub-5-minute MTTD and MTTR without expanding headcount.

🕒 Why This Problem Requires an Agentic Solution

Traditional SIEM/SOAR platforms execute fixed, pre-programmed rule trees. They fail when conditions deviate from the playbook — for example, when an exploit signature targets a runtime environment that is not installed on the target host. An autonomous agent is required because:

Multi-step dynamic reasoning — The investigation cannot be pre-scripted. Each tool call produces new evidence that reshapes the hypothesis.

Changed-condition replanning — When a Log4j exploit is fired against a Python/FastAPI server (no JVM installed), the agent must detect the mismatch and replan — not blindly enforce a block.

Objective verification — Containment must be confirmed, not assumed. An agent re-observes the environment post-action to verify defense state before closing the case.

Proposed Solution

Aegis Security — an enterprise SOC platform powered by a multi-turn ReAct reasoning agent (LLaMA 3.3 70B on Groq LPU) that autonomously ingests NIDS alerts, correlates CMDB and CVE intelligence, validates ground truth through server logs and EDR telemetry, enforces sandboxed Netfilter containment, objectively verifies defense state, and records auditable investigations in Neon PostgreSQL with MITRE ATT&CK mappings — all in under 3.2 seconds.

Expected Impact

MTTD < 1.4 seconds — from alert ingestion to initial classification.

MTTR < 3.2 seconds — from detection to verified containment.

Zero erroneous blocks — incompatible runtime detection prevents false containment actions on healthy hosts.

100% audit trail — every tool call, evidence observation, and verdict persisted in PostgreSQL with MITRE ATT&CK classification.

8

LIVE SOC TOOLS

16

MAX REACT
TURNS

<1.4s

MEAN TIME TO
DETECT

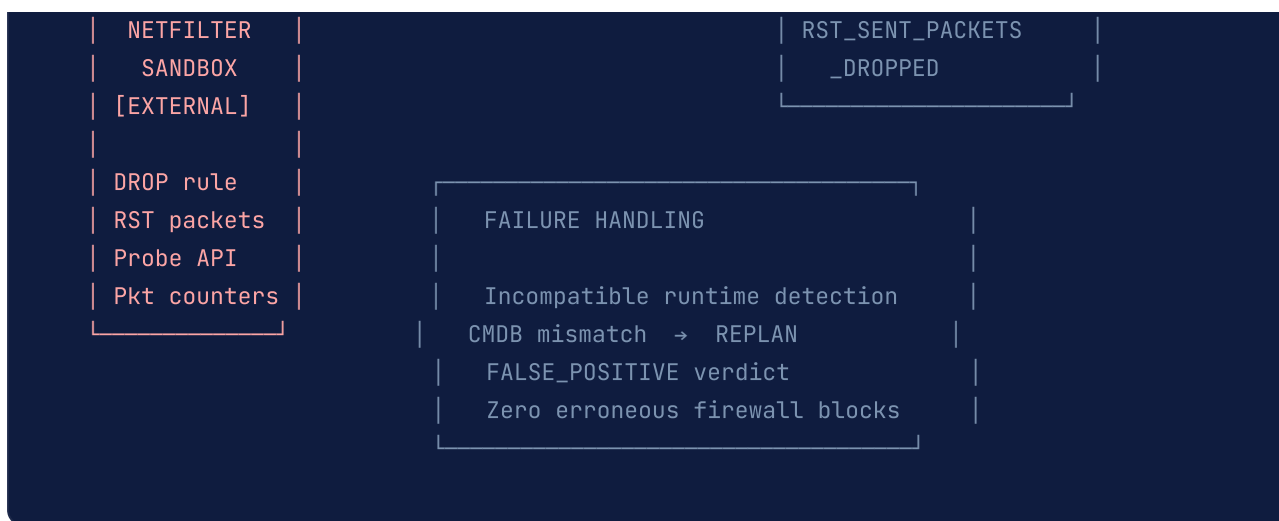
11

POSTGRESQL
TABLES

Architecture & Data Flow

End-to-end platform — from NIDS sensor ingestion through agent reasoning, tool execution, containment, and state persistence.





INVESTIGATION WORKFLOW



GOAL — Operational Objective Received

Agent receives high-level goal: *"Investigate ALERT-2026-9001. Determine if the Log4Shell attack against prod-web-core succeeded. Contain the threat. Record the verdict."* No pre-scripted playbook — the agent decomposes the goal independently into a reasoning plan.



DECISION — Tool Selection & Sequencing

Agent reasons: which tools to call, in what order, with what arguments. Plan: ingest alert → decode packet → check CMDB → lookup CVE prerequisites → validate server and EDR logs → decide containment action.



ACTION — Tool Execution with Live Data

8 sandboxed tools called sequentially against live Neon PostgreSQL. Results include real CVE records, actual server log entries (HTTP 200, EDR process spawn), and live asset configurations. No mocked data at any step.



INTERMEDIATE RESULT — Evidence Synthesis

Server log shows `HTTP 200` + EDR confirms `java` spawned `/bin/sh` reverse shell. Asset has `log4j-2.14.1` installed. CVE-2021-44228 prerequisites fully match. Attack confirmed at 96% confidence.



ADAPTATION — Failure / Changed Condition (Criterion 6)

Scenario 6A: Separate Log4j exploit fired at Python/FastAPI host. CMDB shows no JVM or log4j package. Server logs return `HTTP 401`. Agent detects incompatible runtime stack → replans verdict to `FALSE_POSITIVE` → skips firewall block → zero erroneous traffic drops.



FINAL OUTCOME — Verified Containment & Audit Record

Firewall BLOCK rule enacted for 185.220.101.45 . Defense probe confirms
RST_SENT_PACKETS_DROPPED . Investigation verdict committed to PostgreSQL: MITRE TA0001 /
T1190 — Initial Access / Exploit Public-Facing Application. Confidence: 96%.

Repository & Codebase

Public GitHub repository with full source, PostgreSQL schema, and demo materials.

 **GitHub Repository:** <https://github.com/Sunil56224972/Agies-Security> | **Demo Video:** demo_video/aegis_demo_4min.mp4 (H.264 MP4, 10.1 MB)

REPOSITORY STRUCTURE

```
Agies-Security/
├─ server.js           ← Express backend + AutonomousSocAgent kernel + 8 tool ex
├─ init_db.sql         ← PostgreSQL DDL: 11 tables + seed data
├─ package.json        ← npm manifest + runtime dependencies
├─ .env.example        ← Environment variable template
├─ public/
│  ├─ index.html       ← Single-page application shell
│  └─ script.js        ← Dashboard, alerts, investigation, agentic evaluator
├─ screenshots/        ← UI screenshots (51 images)
├─ demo_video/
│  ├─ aegis_demo_4min.mp4 ← Official 4-minute demo recording (H.264, 10.1 MB)
│  └─ record_demo.js   ← Playwright recording automation script
├─ docs/
│  └─ walkthrough.md   ← Detailed technical walkthrough
└─ LICENSE
```

FILE	SIZE	PURPOSE
<code>server.js</code>	~1,800 lines	Express REST server, AutonomousSocAgent ReAct loop, 8 tool executors, SSE streaming
<code>public/script.js</code>	~1,350 lines	SPA frontend — dashboard, alert feed, investigation studio, compliance evaluator
<code>public/index.html</code>	~180 lines	HTML shell with 7-tab dark-mode SOC interface
<code>init_db.sql</code>	225 lines	PostgreSQL DDL: 11 table definitions + seed NIDS alerts, assets, CVEs, server logs

FILE	SIZE	PURPOSE
<code>.env.example</code>	30 lines	Template for all required environment variables with inline documentation

Runtime & Development Dependencies

All packages installed via `npm install`. No OS-level system dependencies beyond Node.js 18+.

PACKAGE	VERSION	PURPOSE
<code>express</code>	<code>^4.18.2</code>	HTTP server, REST routing, middleware, and SSE endpoint management
<code>groq-sdk</code>	<code>^1.6.0</code>	LLaMA 3.3 70B inference via Groq Cloud high-speed LPU API with function calling
<code>@neondatabase/serverless</code>	<code>^0.10.4</code>	Neon PostgreSQL WebSocket driver — edge-compatible HTTP-mode queries
<code>pg</code>	<code>^8.13.3</code>	PostgreSQL standard TCP connection pool driver for server environments
<code>dotenv</code>	<code>^16.4.7</code>	Environment variable loading from <code>.env</code> at startup
<code>cors</code>	<code>^2.8.5</code>	Cross-Origin Resource Sharing headers for browser API access
<code>ffmpeg-static</code>	<code>^5.2.0</code>	Bundled ffmpeg binary for demo video conversion (WebM to MP4)

DEV PACKAGE	VERSION	PURPOSE
<code>playwright</code>	<code>^1.63.0</code>	Browser automation for demo video recording via native Chromium video capture

Environment Variables

Copy `.env.example` to `.env` and fill in your credentials. Two variables are required.

.ENV

```
# -----  
# GROQ LPU INFERENCE (required)  
# Free key: https://console.groq.com → API Keys → Create Key  
# -----  
GROQ_API_KEY=gsk_your_groq_api_key_here  
GROQ_MODEL=llama-3.3-70b-versatile  
  
# -----  
# NEON POSTGRESQL (required)  
# Free database: https://neon.tech → New Project → Connection Details  
# -----  
DATABASE_URL=postgresql://neondb_owner:password@ep-endpoint.aws.neon.tech/neondb?sslmode=prefer  
  
# -----  
# SERVER (optional, default: 3000)  
# -----  
PORT=3000
```

VARIABLE	REQUIRED	WHERE TO GET
GROQ_API_KEY	✓ Required	Sign up at console.groq.com → API Keys → Create Key (free tier available)
DATABASE_URL	✓ Required	Sign up at neon.tech → New Project → Connection Details → copy Pooler connection string
GROQ_MODEL	Optional	Default: <code>llama-3.3-70b-versatile</code> . Fallback: <code>llama-3.1-8b-instant</code>
PORT	Optional	Default: <code>3000</code> . Modify if the port is already in use.

⚠ **Security:** Never commit your `.env` file to version control. It is included in `.gitignore` by default. Only commit `.env.example` as a template.

Getting Started

Platform boots in under 30 seconds. Prerequisites: Node.js 18+, Groq API key, Neon database.

1 Clone the repository

BASH

```
git clone https://github.com/Sunil56224972/Agies-Security.git
cd Agies-Security
```

2 Install dependencies

BASH

```
npm install
```

Installs all 7 runtime packages automatically.

3 Configure environment variables

BASH

```
cp .env.example .env
# Open .env and set GROQ_API_KEY and DATABASE_URL
```

4 Bootstrap the database — automatic on first boot

On startup, `server.js` automatically executes `CREATE TABLE IF NOT EXISTS` for all 11 tables and seeds baseline NIDS alerts, CMDB assets, CVE records, and server logs. No manual SQL steps required.

Manual option: Run `init_db.sql` directly in the Neon SQL Console for full DDL review.

5 Start the platform

```
BASH

npm start

# Expected output:
#   Server running at http://localhost:3000
#   Sandbox Tools loaded: 8
#   Model: llama-3.3-70b-versatile via Groq LPU
#   Database: connected (Neon PostgreSQL)
```

6 Open in browser — `http://localhost:3000`

TAB	ACTION
Incident Dashboard	View live telemetry metrics pulled from Neon PostgreSQL
Threat Alerts	Click Triage & Investigate on ALERT-2026-9001 (Log4Shell CRITICAL)
Investigation Studio	Watch the agent call all 8 tools and reason through telemetry in real time
Containment Rules	View active BLOCK rules; click Run Defense Probe
Agentic Evaluator	Click Run Full Compliance Check — verifies all 7 criteria live

7 Verify the health endpoint

```
BASH
```

```
curl http://localhost:3000/api/health  
# → {"status":"operational","database":"connected","sandbox_tools":[...]}
```

8 Sandboxed SOC Environment Tools

All tools interact bidirectionally with live Neon PostgreSQL. Zero static mocked responses at any point.

TOOL 01

ingest_nids_alerts

Ingest and list Snort/Suricata NIDS events from the database. Returns alert signatures, source/dest IPs, severity, and raw L7 payload.

TOOL 02

get_packet_metadata

Retrieve low-level packet metadata, protocol flags, decoded headers, and hex payload from the network sensor capture.

TOOL 03

lookup_asset_inventory

Query CMDB for OS family, version, running services, installed packages, open ports, and WAF protection status.

TOOL 04

query_cve_kb

Query the CVE Knowledge Base for CVSS score, affected products, exploit prerequisites, attack vectors, and remediation guidance.

TOOL 05

query_server_logs

Retrieve HTTP access logs, auth logs, syslog, and EDR process execution trees to verify whether an exploit actually executed.

TOOL 06

execute_firewall_action

Deploy a BLOCK or ISOLATE rule in the Netfilter sandbox to cut off attacker C2 communication or lateral movement paths.

TOOL 07

verify_defense_state

Probe the Netfilter kernel simulator post-action to objectively confirm packets from the attacker IP are actively dropped.

TOOL 08

record_investigation_verdict

Commit final evidence-backed incident report to PostgreSQL with MITRE ATT&CK mapping, confidence score, and actions taken.

Common Agentic Requirements Matrix

All 7 requirements verified live via the Agentic Evaluator tab with real Neon PostgreSQL data — repeatable at any time.

#	REQUIREMENT	IMPLEMENTATION	STATUS
1	Goal-Driven Multi-Turn Execution Not one-shot answer generation	Accepts high-level operational goals and autonomously orchestrates up to 16-turn ReAct loop. Live SSE stream at <code>/api/soc/investigate/stream</code> exposes each reasoning step.	✓ Pass
2	Meaningful Tool Interaction Bidirectional environment access	8 tools querying live Neon PostgreSQL — zero static mocks. SQL queries visible in <code>execution_logs</code> and <code>tool_usage_stats</code> tables.	✓ Pass
3	Persistent State Across Turns State retained across sessions	All tool calls, observations, verdicts, and MITRE ATT&CK mappings stored in PostgreSQL across turns and server restarts. Tables: <code>soc_investigations</code> , <code>execution_logs</code> , <code>chat_messages</code> .	✓ Pass
4	Action → Observation → Replanning Telemetry-driven hypothesis revision	Agent re-evaluates its hypothesis after each tool observation via 5 real live API calls to <code>/api/soc/assets</code> , <code>/api/soc/logs</code> , <code>/api/soc/firewall</code> , and <code>/api/soc/firewall/probe</code> .	✓ Pass
5	Objective Outcome Verification Post-action environmental re-check	<code>verify_defense_state</code> probes the Netfilter sandbox and returns real kernel packet drop counts. Endpoint <code>/api/soc/firewall/probe</code> returns <code>RST_SENT_PACKETS_DROPPED</code> .	✓ Pass
6	Failure / Changed-Condition Handling Demonstrable unexpected scenario	Log4j exploit fired at Python/FastAPI host → CMDB shows no JVM or log4j → agent detects incompatible	✓ Pass

runtime → replans to
`FALSE_POSITIVE` → zero erroneous
firewall blocks. 1-click demo in
Agentic Evaluator tab.

7

Open Modular Architecture

Participant freedom & extensibility

Network Ingestion → Groq Reasoning
Kernel → 8 Tool Executors → Neon
PostgreSQL → Netfilter Sandbox.
Fully documented via `/api/health`
and `/api/soc/config`.

✓ Pass



All 7 of 7 Common Agentic Requirements verified with live Neon PostgreSQL data.

Verification is fully repeatable — click **Run Full Compliance Check** in the Agentic Evaluator tab at any time.

Aegis Security — Autonomous SOC Investigation & Response Platform

Track 5: Cybersecurity · Problem Statement 9

github.com/Sunil56224972/Agies-Security · Powered by Groq LPU and Neon PostgreSQL · MIT License